JINDAL STEEL

JSPulse OHMS Technical Architecture & Algorithm Flows

A Deep-Dive into the Core Logic, Automation Pipelines, and Codebase Architecture

Developed By: Ashwini Kumar

Intern ID: JSPL-2026-IT-042

Department: Information Technology & Occupational Health Services Group

Host Organization: Jindal Steel

A Deep-Dive into the Core Logic, Automation Pipelines, and Codebase Architecture

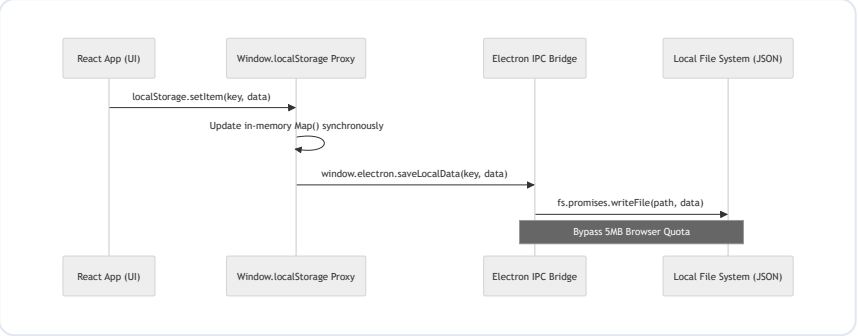
1. Custom LocalStorage Proxy Engine & 5MB Quota Bypass

Location: `src/main.jsx` | `electron/main.js`

Overview

Browser `localStorage` is strictly capped at 5MB, which is insufficient for an enterprise healthcare system handling patient logs and Excel grids. JSPulse OHMS bypasses this limitation by injecting a `Proxy` object that intercepts all `setItem` and `getItem` calls. It caches data synchronously in-memory for React UI rendering, while asynchronously flushing the payloads to the native Windows file system (`C:\OHSData`) via Electron IPC.

Flowchart



Implementation Code

```
// src/main.jsx - The Storage Proxy Interceptor
const memoryStorage = new Map();

const storageProxy = new Proxy(window.localStorage, {
  getItem: function(target, prop) {
    if (typeof target[prop] === 'function') return target[pr
    if (memoryStorage.has(prop)) return memoryStorage.get(pr
    return target.getItem(prop);
  },
  setItem: function(target, prop, value) {
    memoryStorage.set(prop, value);
    // Bypass quota by routing to Node.js FS
    if (window.electron && window.electron.saveLocalData) {
      window.electron.saveLocalData(prop, value).catch(err =
        console.error('IPC sync failed', err)
      );
    }
    return true;
  }
});
```

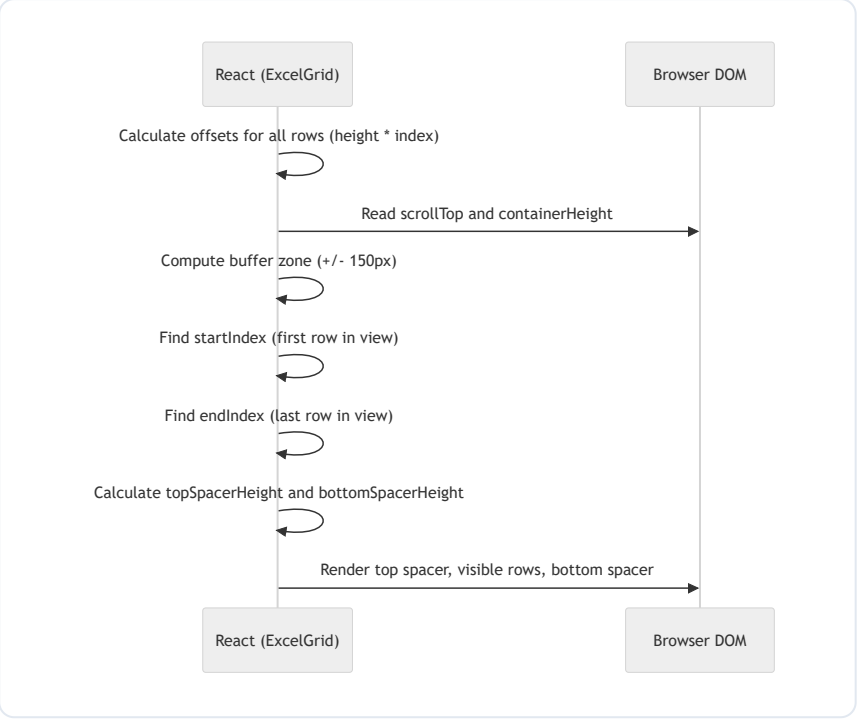
2. DOM Viewport Virtualization Algorithm

Location: `src/components/clinical/ExcelGrid.jsx`

Overview

Rendering 10,000+ spreadsheet cells simultaneously crashes the browser's paint engine. The `ExcelGrid` component implements a DOM virtualization algorithm. It calculates the user's scroll position and dynamically renders only the rows visible within the viewport, using "spacer" divs above and below to maintain the correct scrollbar height.

Flowchart



Implementation Code

```
// src/components/clinical/ExcelGrid.jsx
const visibleRows = useMemo(() => {
    let start = 0;
    let end = data.length - 1;
    let topSpacerHeight = 0;
    let bottomSpacerHeight = 0;

    const buffer = 150;
    let currentOffset = 0;

    for (let i = 0; i < data.length; i++) {
      const rowHeight = rowHeights[i] || 30;
      if (currentOffset + rowHeight < scrollTop - buffer) {
        start = i + 1;
        topSpacerHeight += rowHeight;
      } else if (currentOffset > scrollTop + containerHeight
        end = i - 1;
        break;
      }
      currentOffset += rowHeight;
    }

    if (end < start) end = start;

    // Calculate remaining height for bottom spacer
    for (let i = end + 1; i < data.length; i++) {
        bottomSpacerHeight += (rowHeights[i] || 30);
    }

    return { start, end, topSpacerHeight, bottomSpacerHeight
}, [data, rowHeights, scrollTop, containerHeight]);
```

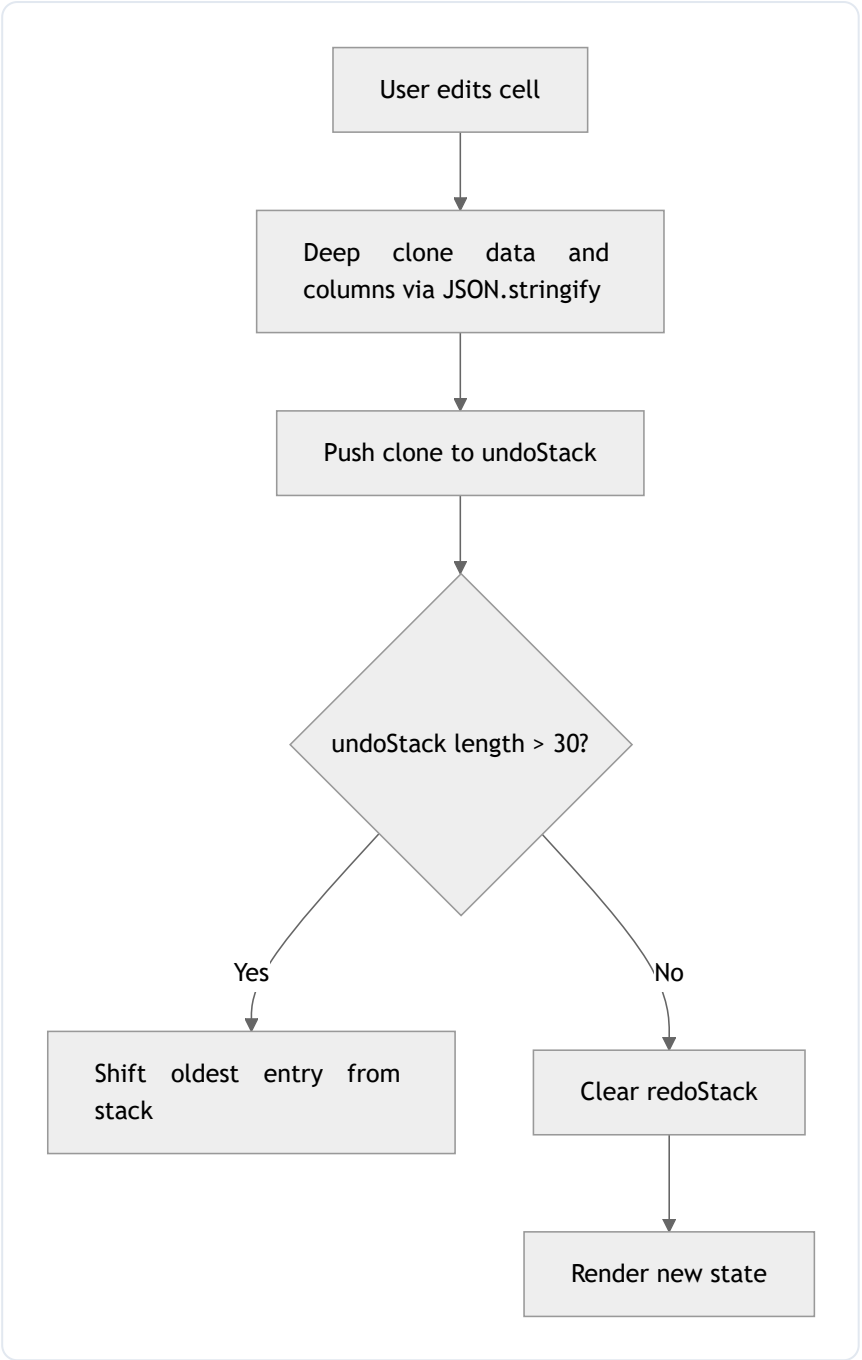
3. Undo/Redo State History Stacks

Location: `src/components/clinical/ExcelGrid.jsx`

Overview

To provide a native Excel feel, the application maintains isolated memory stacks for Undo and Redo operations. Deep cloning is utilized to prevent object reference mutation. The stack is capped at 30 operations to prevent memory leaks in the browser heap.

Flowchart


User edits cell
Deep clone data and columns via JSON.stringify
Push clone to undoStack
undoStack length > 30?
Yes
No
Shift oldest entry from stack
Clear redoStack
Render new state

Implementation Code

```
// src/components/clinical/ExcelGrid.jsx
const pushUndo = (newData, newCols) => {
    setUndoStack(prev => {
        // Deep clone to prevent reference bridging
        const newState = {
            data: JSON.parse(JSON.stringify(newData)),
            columns: JSON.parse(JSON.stringify(newCols))
        };
        const newStack = [...prev, newState];
        if (newStack.length > 30) newStack.shift(); // Cap m
        return newStack;
    });
    setRedoStack([]); // Invalidate future history on new ac
};
```

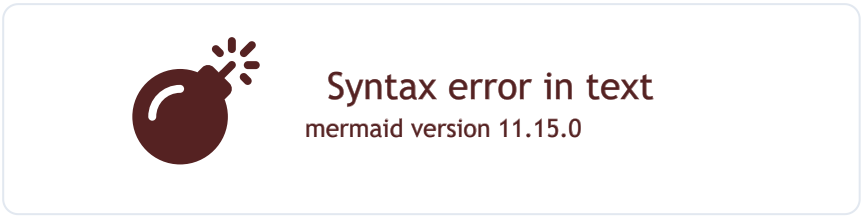
4. Spreadsheet Formula Evaluation Engine

Location: `src/components/clinical/ExcelGrid.jsx`

Overview

The spreadsheet grid supports custom formulas like `=SUM(A1:B5)` or `=A1+B2`. The evaluation engine utilizes recursive regex parsing to map alpha-numeric cell coordinates (e.g., 'A1') to matrix indices, evaluating them dynamically. It includes a depth limiter to prevent infinite loop crashes on circular references.

Flowchart



Implementation Code

```
// src/components/clinical/ExcelGrid.jsx
const evaluateFormula = (val, dataRef, depth = 0) => {
    if (depth > 50) return 0; // Prevent circular reference
    let formula = val.substring(1).toUpperCase();

    // Parse SUM(A1:B2) ranges
    formula = formula.replace(/SUM\(([A-Z]+)(\d+):([A-Z]+)(\
        let sum = 0;
        const colStart = colLetterToIndex(c1);
        const colEnd = colLetterToIndex(c2);
        const rowStart = parseInt(r1) - 1;
        const rowEnd = parseInt(r2) - 1;

        for (let r = Math.min(rowStart, rowEnd); r <= Math.m
            for (let c = Math.min(colStart, colEnd); c <= Ma
                if (dataRef[r] && dataRef[r][columns[c]?.fie
                    let cellVal = dataRef[r][columns[c].fiel
                    if (typeof cellVal === 'string' && cellV
                        cellVal = evaluateFormula(cellVal, d
                    }
                    const num = parseFloat(cellVal);
                    if (!isNaN(num)) sum += num;
                }
            }
        }
        return sum;
    });
    // ... eval() fallback for standard math
};
```

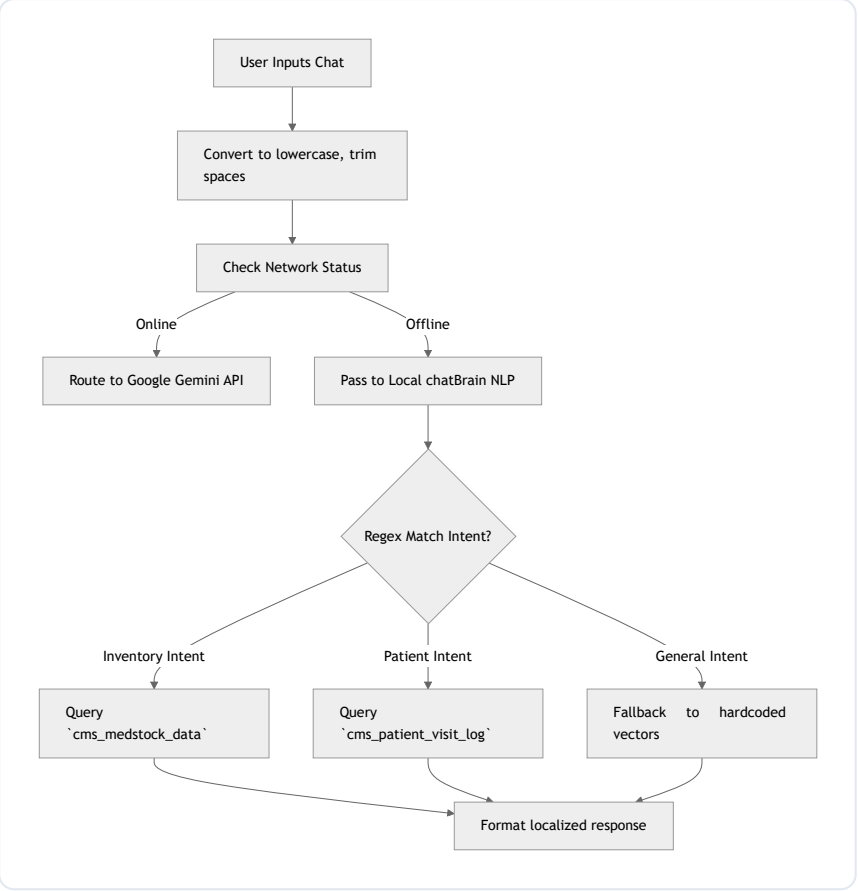
5. Offline NLP Chat Engine & Token Classifier

Location: `src/utils/chatBrain.js`

Overview

JSPulse OHMS can operate entirely offline without Google Gemini if internet connectivity is lost. It utilizes a custom Natural Language Processing (NLP) token classifier that scans input arrays against localized trigger vectors to determine intent, querying local databases for answers.

Flowchart



Implementation Code

```
// src/utils/chatBrain.js
export const processChatMessage = async (message) => {
    const lowerMsg = message.toLowerCase();

    // Patient Lookup Token Classifier
    if (lowerMsg.includes('patient') || lowerMsg.includes('v
        const patients = getStorageData('cms_patient_visit_l
        if (patients.length === 0) return "There are no pati
        return `I found ${patients.length} patient visits in
    }

    // Inventory Lookup Token Classifier
    if (lowerMsg.includes('stock') || lowerMsg.includes('med
        const stock = getStorageData('cms_medstock_data') ||
        const lowStock = stock.filter(item => parseInt(item.

        if (lowStock.length > 0) {
            return `You have ${stock.length} medicines in in
        }
        return `You have ${stock.length} medicines in invent
    }
};
```

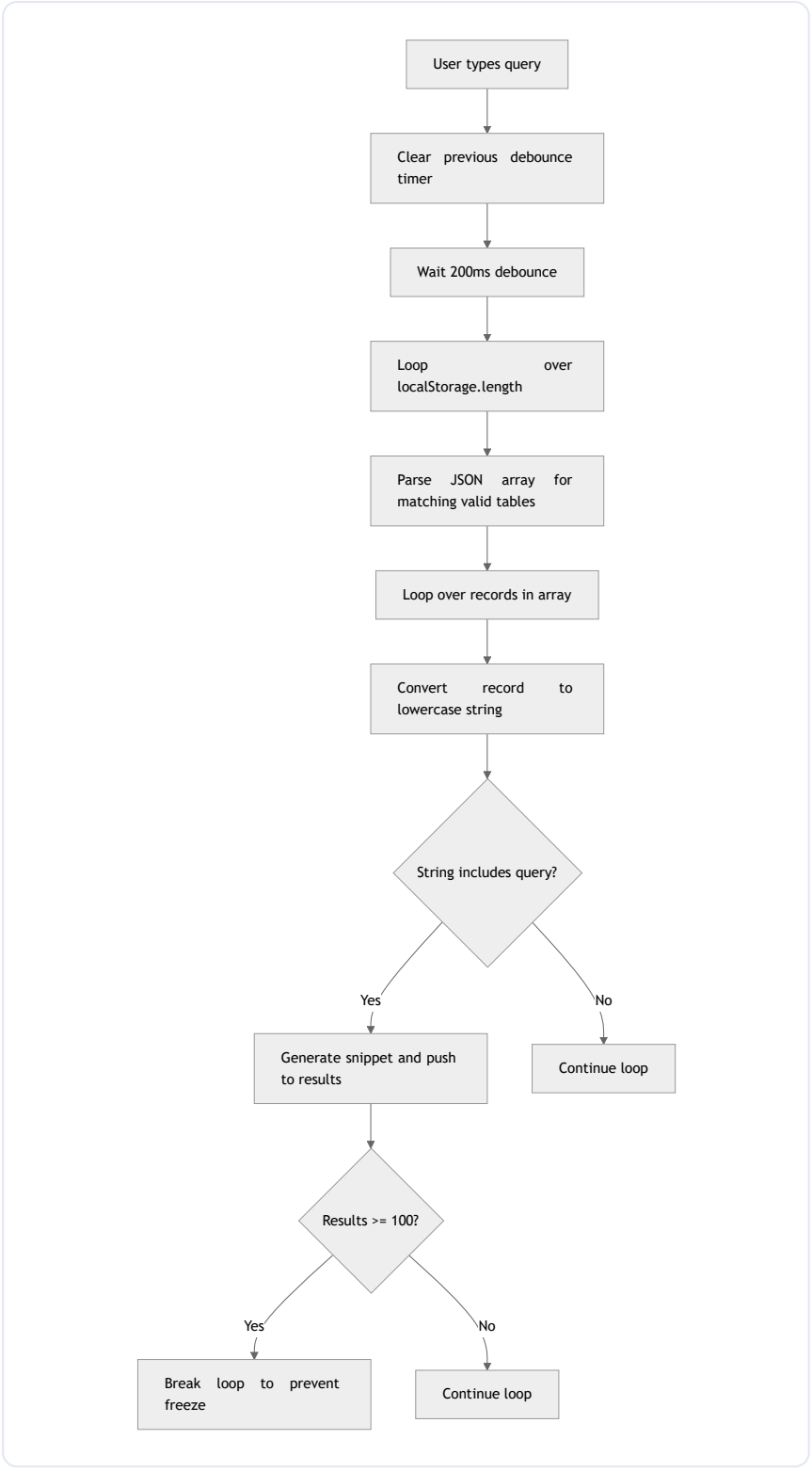
6. Global System Search Indexer

Location: `src/components/GlobalSearch.jsx`

Overview

The `CTRL+K` global search must scan through every schema across the application simultaneously. To prevent main thread blocking, it utilizes a debounced polling loop that iterates through the `localStorage` key index, parses JSON arrays on-the-fly, and stops if the results exceed 100 to prevent DOM freezing.

Flowchart


User types query
Clear previous debounce timer
Wait 200ms debounce
Loop over localStorage.length
Parse JSON array for matching valid tables
Loop over records in array
Convert record to lowercase string
String includes query?
Yes
No
Generate snippet and push to results
Continue loop
Results >= 100?
Yes
No
Break loop to prevent freeze
Continue loop

Implementation Code

```
// src/components/GlobalSearch.jsx
debounceRef.current = setTimeout(() => {
    const searchTerm = term.toLowerCase();
    const foundResults = [];
    const MAX_RESULTS = 100;

    for (let i = 0; i < localStorage.length; i++) {
        if (foundResults.length >= MAX_RESULTS) break;
        const key = localStorage.key(i);
        const rawData = localStorage.getItem(key);

        if (rawData) {
            const parsedData = JSON.parse(rawData);
            if (Array.isArray(parsedData)) {
                for (let ri = 0; ri < parsedData.length; ri+
                    const record = parsedData[ri];
                    const recordString = Object.values(recor
                        .filter(val => typeof val !== 'objec
                        .map(val => String(val).toLowerCase(
                        .join(' ');

                    if (recordString.includes(searchTerm)) {
                        foundResults.push({ id: `${key}-${ri
                    }
                }
            }
        }
    }
    setResults(foundResults);
}, 200);
```

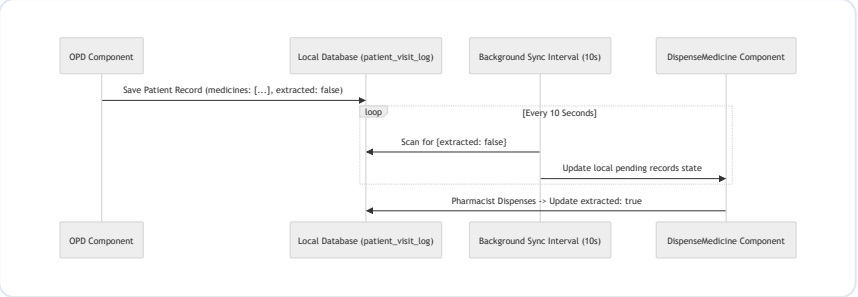
7. Automated OPD-to-Pharmacy Sync Pipeline

Location: `src/components/DispenseMedicine.jsx`

Overview

When an OPD doctor inputs a prescription, it must automatically reflect in the pharmacist's dispense queue without manual refresh. A `setInterval` hook polls the `cms_patient_visit_log` every 10 seconds, identifying records where `extracted = false` and copying them into the active dispense array.

Flowchart



Implementation Code

```
// src/components/DispenseMedicine.jsx
const extractFromOPD = () => {
    const opdData = getStorageData('cms_patient_visit_log')
    const dispenseData = getStorageData('cms_dispense_log')
    let updated = false;

    opdData.forEach(opdRecord => {
        // Extract if medicines exist and it hasn't been ext
        if (opdRecord.medicines && opdRecord.medicines.lengt

            const newDispenseRecord = {
                id: `disp-${Date.now()}-${Math.random().toSt
                date: opdRecord.date,
                patientName: opdRecord.patientName,
                empId: opdRecord.empId,
                medicines: opdRecord.medicines,
                status: 'Pending',
                source: 'OPD'
            };

            dispenseData.push(newDispenseRecord);
            opdRecord.extracted = true;
            updated = true;
        }
    });

    if (updated) {
        saveStorageData('cms_patient_visit_log', opdData);
        saveStorageData('cms_dispense_log', dispenseData);
    }
};
```

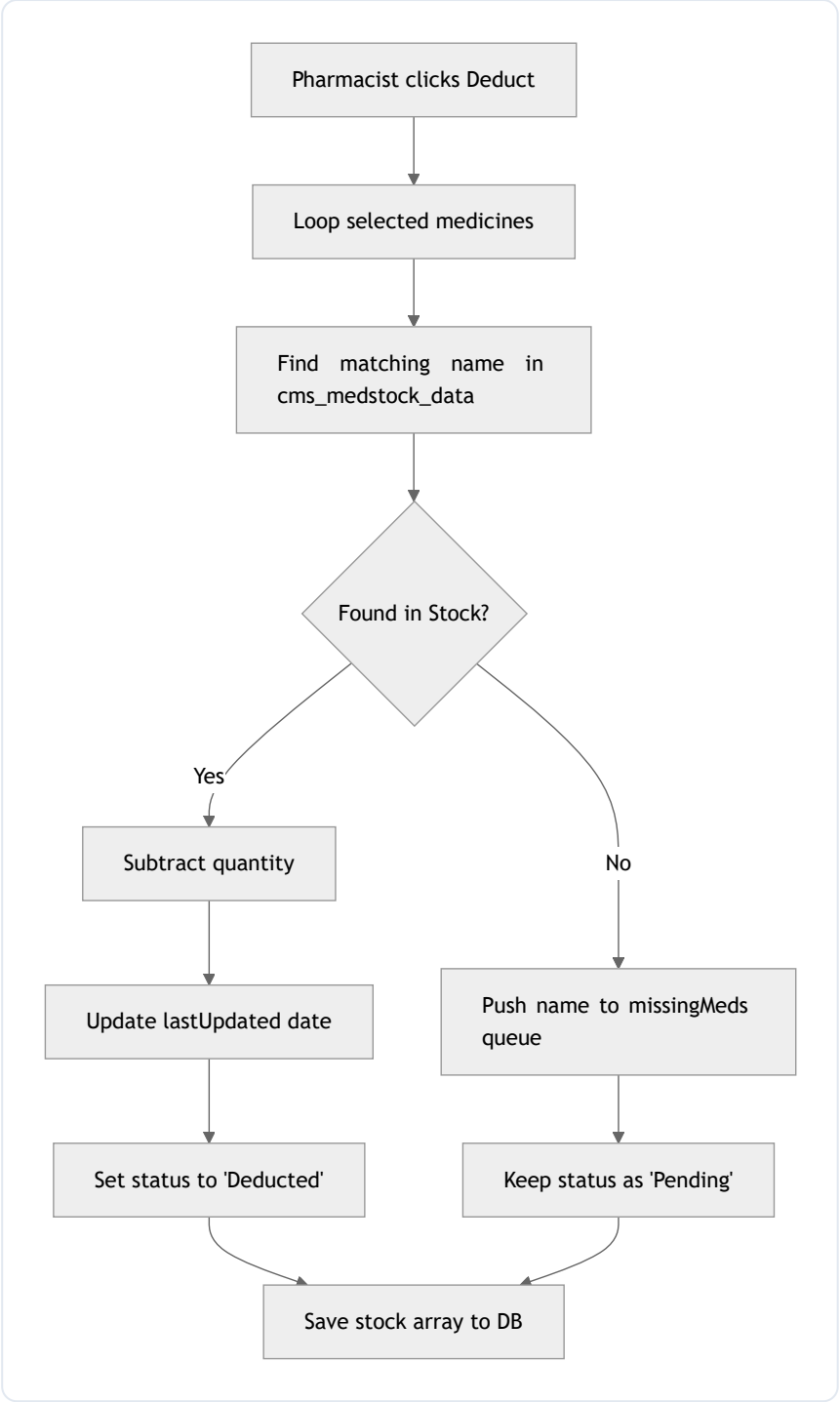
8. Out-of-Stock Resiliency & Inventory Deduction

Location: `src/components/DailyConsume.jsx`

Overview

When stock is consumed, the system maps the provided medicine names against the master stock database (`cms_medstock_data`). If a drug is found, it decrements the `currentStock`. If the stock is mismatched or missing, the system catches the failure gracefully, routing the drug to a `missingMeds` queue to alert the pharmacist without halting the entire transaction.

Flowchart


Pharmacist clicks Deduct
Loop selected medicines
Find matching name in cms_medstock_data
Found in Stock?
Yes
No
Subtract quantity
Update lastUpdated date
Set status to 'Deducted'
Push name to missingMeds queue
Keep status as 'Pending'
Save stock array to DB

Implementation Code

```
// src/components/DailyConsume.jsx
const updatedStock = [ ... stockData];
const missingMeds = [];

record.medicines.forEach(med ⇒ {
    // Attempt strict string match
    const stockIndex = updatedStock.findIndex(
        s ⇒ s.medicineName.toLowerCase() ⩵ med.name.toLow
    );

    if (stockIndex ≠ -1) {
        const currentStock = parseInt(updatedStock[stockInde
        const consumeQty = parseInt(med.qty) || 0;
        // Deduct inventory
        updatedStock[stockIndex].currentStock = Math.max(0,
        updatedStock[stockIndex].lastUpdated = new Date().to
    } else {
        // Fallback: Medicine not found in master stock
        missingMeds.push(med.name);
    }
});

if (missingMeds.length > 0) {
    alert(`Could not deduct stock for: ${missingMeds.join(',
}
```

9. Storage Quota Management & Auto-Migration

Location: `src/utils/SystemHealthTool.jsx` & `backupDB.js`

Overview

Browsers strictly enforce limits on `localStorage`. The OHMS calculates byte size using UTF-16 encoding principles (2 bytes per character). When limits are approached, background utilities compress older logs and automatically migrate blob data over to IndexedDB, clearing the active `localStorage` heap.

Implementation Code

```
// src/components/clinical/SystemHealthTool.jsx
const estimateLocalStorageQuota = () => {
    let _lsTotal = 0, _xLen, _x;
    for (_x in localStorage) {
        if (!localStorage.hasOwnProperty(_x)) continue;
        _xLen = ((localStorage[_x].length + _x.length) * 2);
        _lsTotal += _xLen;
    }
    const usageMB = (_lsTotal / (1024 * 1024)).toFixed(2);
    // Average browser limit is 5MB
    const totalMB = 5.0;
    const percent = ((usageMB / totalMB) * 100).toFixed(1);

    return { usageMB, totalMB, percent };
};
```

10. Electron-Safe Excel Exporter Engine

Location: `src/utils/exportUtils.js`

Overview

Standard Excel export libraries rely on browser DOM downloads. Because JSPulse OHMS operates in a native Electron wrapper, it bypasses DOM blob links, utilizing `xlsx` to construct array buffers, and then bridging IPC to Node.js for a native Windows "Save As..." system dialog.

Implementation Code

```
// src/utils/exportUtils.js
import * as XLSX from 'xlsx';

export const exportToExcel = (data, filename) => {
    const ws = XLSX.utils.json_to_sheet(data);
    const wb = XLSX.utils.book_new();
    XLSX.utils.book_append_sheet(wb, ws, "Data");

    // Write to array buffer
    const excelBuffer = XLSX.write(wb, { bookType: 'xlsx', t

    // If in Electron, use native IPC save dialog bypassing
    if (window.electron && window.electron.saveExcelFile) {
        window.electron.saveExcelFile(excelBuffer, filename)
            .then(res => {
                if(res.success) alert(`Saved to: ${res.fileP
            });
        return;
    }
};
```

11. Electron Headless PDF Print Bridge

Location: `electron/main.js`

Overview

When the user generates a report, it must contain rendered Mermaid.js diagrams. Electron intercepts the IPC `print-to-pdf` call, injects a headless Chromium `webContents` instance, waits 2500ms for SVGs to mount, and then flushes the buffer directly to a .pdf file on the native file system.

Implementation Code

```
// electron/main.js
ipcMain.handle('print-to-pdf', async (event, htmlContent, de
  return new Promise((resolve) => {
    let win = new BrowserWindow({ show: false });
    win.loadURL(`data:text/html;charset=utf-8,${encodeURICom

    win.webContents.on('did-finish-load', () => {
      // Allow Mermaid.js CDN to render SVG nodes before pri
      setTimeout(() => {
        win.webContents.printToPDF({
          printBackground: true,
          pageSize: 'A4',
          marginType: 0 // No margins
        }).then(data => {
            const tempPath = path.join(os.tmpdir(), 'report.
            fs.writeFileSync(tempPath, data);

            dialog.showSaveDialog({ defaultPath: defaultPath
                if (!result.canceled) {
                    fs.copyFileSync(tempPath, result.filePat
                    resolve({ success: true, filePath: resul
                }
                win.close();
            });
        });
      }, 2500);
    });
  });
});
```

12. Complete System Summary & Macro Architecture

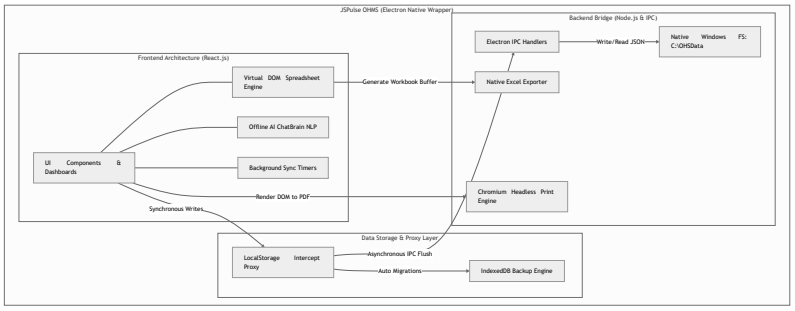
Project: JSPulse OHMS (Occupational Health Management System)

Executive Summary

JSPulse OHMS is an advanced, offline-first enterprise healthcare registry designed entirely around the constraints of restricted network environments. Built with a React.js frontend and wrapped in a native Electron native container, the platform bridges the gap between modern web technologies and secure, local file-system operations.

By utilizing a completely custom `Proxy` interception layer on top of browser `localStorage`, the system dynamically routes all state changes to the local Windows file system (`C:\OHSData`) and IndexedDB. This architecture bypasses standard 5MB browser quotas while maintaining millisecond read/write speeds for massive datasets (like 10,000+ row spreadsheet renders). The system heavily automates daily clinical pipelines, featuring automated background synchronization hooks that seamlessly poll and push data between the OPD prescribing doctors and the Pharmacy dispensing queues. When offline, a localized Natural Language Processing (NLP) token classifier steps in to replace cloud APIs, allowing staff to query patient visits and medical stock inventories using conversational English.

Complete Software Architecture Flowchart



End of Technical Architecture Document.